



WEATHER FIT TALK — 발표 대본

총 18슬라이드 | 예상 발표 시간: 약 12~15분

슬라이드 01 · 표지

⌚ 예상 시간: 30초

안녕하세요, 박수용입니다.

오늘 제가 발표할 프로젝트는 **WEATHER FIT TALK**입니다.

한 줄로 정리하면, "오늘 날씨와 상황에 맞는 코드를 AI가 추천해주는 챗봇"입니다.

Python과 Streamlit으로 웹 화면을 만들고, Ollama라는 도구로 AI 모델을 제 컴퓨터에서 직접 실행했습니다. API 비용 없이, 인터넷 없이도 돌아가는 로컬 AI 서비스입니다.

바로 시작하겠습니다.

➡ **[다음 슬라이드로]** 오늘 발표는 크게 네 챕터로 구성되어 있습니다.

슬라이드 02 · 발표 개요

⌚ 예상 시간: 30초

오늘 발표는 네 파트로 진행됩니다.

첫 번째는 **INTRO**, 프로젝트가 무엇이고 왜 만들었는지입니다. 두 번째는 **STRUCTURE**, 전체 구조와 사용 기술입니다. 세 번째는 **CODE**, 핵심 파일 세 개를 직접 설명드립니다. 네 번째는 **CLOSING**, 통신 방식과 구현 과정, 그리고 마무리입니다.

개발을 잘 모르셔도 이해할 수 있도록 비유와 예시를 최대한 활용해서 설명하겠습니다.

➡ **[다음 슬라이드로]** 먼저 이 프로젝트가 정확히 무엇인지 한 줄로 소개드리겠습니다.

슬라이드 03 · 한 줄 소개

🕒 예상 시간: 1분

이 프로젝트의 핵심은 딱 한 문장입니다.

"오늘 날씨와 상황에 맞는 코드를 **AI**가 추천해주는 챗봇."

슬라이드 오른쪽에 보이는 것처럼, 사용자가 이렇게 입력합니다.

"오늘 비 오고 바람이 불어. 학교 갈 때 뭐 입을까?"

그러면 **AI**가 이렇게 답합니다.

"방수 바람막이 + 어두운 데님 + 미끄럼방지 스니커즈. 작은 우산까지 챙기시면 좋을 것 같아요."

상의, 하의, 아우터, 신발, 소품까지 한 번에 추천해줍니다.

날씨 앱처럼 기온만 알려주는 게 아니라, 실제로 오늘 뭘 입을지를 말해주는 서비스입니다.

➡ **[다음 슬라이드로]** 그렇다면 왜 이런 프로젝트를 만들었는지 배경을 말씀드리겠습니다.

슬라이드 04 · 왜 만들었나

🕒 예상 시간: 1분

시작은 아주 단순한 고민이었습니다.

매일 아침 "오늘 뭐 입지?"라는 생각, 다들 한 번씩 해보셨을 겁니다. 날씨 앱을 열어보면 기온과 강수확률은 나오는데, 정작 "그래서 뭘 입어야 해?"는 알려주지 않습니다.

여기서 두 가지 아이디어가 생겼습니다.

첫 번째, **AI**라면 날씨, 상황, 스타일을 한 번에 고려해서 사람이 직접 골라주는 것처럼 추천할 수 있지 않을까?

두 번째, 이 참에 내 컴퓨터에서 직접 돌아가는 **AI**, 즉 로컬 **LLM**의 구조를 직접 만들어보면서 이해하고 싶었습니다.

이 두 가지 동기에서 **WEATHER FIT TALK**가 시작됐습니다.

➡ **[다음 슬라이드로]** 그럼 이 프로젝트에서 실제로 어떤 기능들이 작동하는지 살펴보겠습니다.

슬라이드 05 · 핵심 기능 8가지

🕒 예상 시간: 1분

사용자가 한 마디를 입력하면 뒤에서 **8**가지 기능이 동시에 작동합니다.

하나씩 간단히 짚어드리겠습니다.

F01 날씨 기반 코디 추천 — 기온, 바람, 강수 조건을 보고 옷차림을 자동으로 제안합니다.

F02 상황별 추천 — 등교, 발표, 면접, 데이트처럼 **TPO**, 즉 상황에 맞게 추천이 달라집니다.

F03 스타일 선택 — 미니멀, 캐주얼, 스트릿, 포멀 중 원하는 스타일을 고를 수 있습니다.

F04 사용자 입력 우선 반영 — 사이드바에서 설정한 값보다 직접 쓴 문장을 더 먼저 적용합니다. 이 부분이 이 프로젝트의 핵심 설계 포인트 중 하나입니다.

F05 Ollama 로컬 **LLM** — 인터넷이나 **API Key** 없이 제 **PC**에서 직접 **AI**가 실행됩니다.

F06 DuckDuckGo 검색 — 선택하면 실시간 검색 결과까지 참고해서 더 풍부한 답변을 만듭니다.

F07 오프라인 실행 — 인터넷 없이도 바로 동작합니다.

F08 Streamlit 웹 **UI** — 설치 후 브라우저에서 바로 쓸 수 있는 웹 화면입니다.

➡ **[다음 슬라이드로]** 이 기능들을 구현하는 데 어떤 기술을 사용했는지 설명드리겠습니다.

슬라이드 06 · 기술 스택

🕒 예상 시간: 1분

사용한 기술은 7가지이고, 전부 무료 오픈소스입니다.

Python — 전체 로직을 작성한 프로그래밍 언어입니다. 초보자에게 가장 친숙한 언어라 선택했습니다.

Streamlit — 파이썬 코드만으로 웹 화면을 만들 수 있는 도구입니다. HTML, CSS를 거의 몰라도 웹 앱이 나옵니다.

Ollama — 핵심입니다. 내 컴퓨터에서 AI 모델을 직접 실행할 수 있게 해주는 프로그램입니다. 쉽게 말하면 ChatGPT 서버를 내 PC 안에 설치하는 것과 비슷합니다.

Gemma 4:e4b — 구글이 공개한 오픈소스 AI 모델입니다. 실제로 답변을 만들어내는 두뇌 역할을 합니다.

DuckDuckGo (ddgs) — 필요할 때 검색 결과를 가져와 AI 답변에 활용합니다.

requests — Ollama 서버에 질문을 보내고 답을 받아오는 통신 도구입니다.

CSS — 웹 화면 디자인을 담당합니다.

➡ **[다음 슬라이드로]** 이 기술들이 실제로 어떤 순서로 연결되어 작동하는지, 전체 흐름을 보여드리겠습니다.

슬라이드 07 · 전체 동작 흐름

🕒 예상 시간: 1분 30초

사용자가 질문 한 줄을 입력하면 답변이 나오기까지 총 8단계를 거칩니다.

하나씩 따라가 보겠습니다.

01 사용자가 채팅창에 질문을 입력합니다.

02 Streamlit 웹 앱, 즉 `app.py`가 그 입력을 받습니다.

03 `tools.py`가 사용자 문장에서 날씨와 스타일 힌트를 자동으로 추론합니다. "비 오고 바람이 불어"라는 말에서 날씨 조건을 감지하는 식입니다.

04 사이드바에서 선택한 설정값과 사용자가 직접 입력한 내용을 합칩니다. 이때 사용자가 직접 말한 내용이 항상 우선입니다.

05 DuckDuckGo 검색을 사용하기로 했다면 여기서 검색을 실행합니다.

06 `prompts.py`가 8개 레이어로 구성된 최종 명령문을 만듭니다. 이것을 AI에게 전달할 지시서라고 보시면 됩니다.

07 그 지시서를 Ollama 로컬 LLM에 보내고 AI가 답변을 생성합니다.

08 답변을 정리해서 화면에 출력합니다.

이 전체 흐름의 핵심은 하나입니다. 사용자의 한 문장이 `tools.py`에서 정리되고, `prompts.py`의 규칙에 실려 Ollama LLM으로 전달된다는 것입니다.

➡ **[다음 슬라이드로]** 이 흐름을 담당하는 파일들이 어떻게 구성되어 있는지 구조를 보여드리겠습니다.

슬라이드 08 · 파일 구조

🕒 예상 시간: 45초

프로젝트 파일은 총 6개입니다. 그 중 핵심은 3개입니다.

app.py — 웹 화면, 사용자 입력, 사이드바, Ollama 호출을 모두 담당하는 메인 파일입니다. 레스토랑으로 비유하면 홀 직원 역할입니다.

prompts.py — AI가 어떤 말투와 형식으로 답할지 지시하는 규칙 모음입니다. 요리사에게 주는 레시피북과 같습니다.

tools.py — 날씨와 스타일 추천, 검색 결과 정리를 담당하는 보조 도구 함수들입니다. 재료 손질 도구함 역할입니다.

나머지 `style.css`는 디자인, `requirements.txt`는 설치 목록, `README.md`는 설명서입니다.

➡ **[다음 슬라이드로]** 이제 핵심 파일 세 개를 하나씩 자세히 살펴보겠습니다. 먼저 `app.py`입니다.

슬라이드 09 · app.py 설명

🕒 예상 시간: 1분

`app.py`는 사용자와 AI가 만나는 메인 파일입니다.

이 파일이 하는 일을 순서대로 정리하면 이렇습니다.

01 입력 — 왼쪽 사이드바에서 지역, 날씨, 기온, 상황, 스타일을 선택합니다.

02 입력 — 채팅창에 질문을 한 줄 씩 씁니다.

03 처리 — 입력값을 모아서 AI에게 전달할 프롬프트를 생성합니다.

04 AI 호출 — 완성된 프롬프트를 Ollama 로컬 LLM에 전달합니다.

05 출력 — AI 답변을 받아서 화면에 깔끔하게 정리해 보여줍니다.

한 마디로, 입력을 받고, 처리하고, AI를 부르고, 결과를 보여주는 전체 흐름의 사령탑입니다.

➡ **[다음 슬라이드로]** 다음은 AI에게 어떻게 지시하는지, `prompts.py`를 살펴보겠습니다.

슬라이드 10 · `prompts.py` 설명

🕒 예상 시간: 1분 30초

`prompts.py`는 AI에게 답변 규칙을 전달하는 지침서입니다.

AI는 아무런 규칙 없이 질문만 받으면 엉뚱한 답을 하거나, 매번 다른 형식으로 대답할 수 있습니다. 이걸 막기 위해 8개의 레이어로 규칙을 나눠서 전달합니다.

SYSTEM LAYER — "너는 AI 스타일리스트야"라고 역할을 정의합니다.

SAFETY LAYER — HTML, CSS 코드를 절대 출력하지 말라고 금지합니다.

USER INPUT PRIORITY — 사이드바 설정보다 사용자가 직접 쓴 문장을 먼저 적용하라고 지시합니다.

STYLE PRIORITY — 사용자가 선택한 스타일을 충실히 반영하라고 지시합니다.

WEB SEARCH LAYER — 검색 결과를 참고하되 어떻게 활용할지 규정합니다.

RULE LAYER — 결론을 먼저 말하고, 현실적으로 추천하라는 작성 규칙입니다.

RECOMMENDATION — 상의, 하의, 신발 등 코디 추천의 기준을 정합니다.

OUTPUT FORMAT — 최종 답변의 형식과 순서를 고정합니다.

이 8개 레이어 덕분에 매번 비슷한 톤으로, 같은 형식의 코디 답변이 나옵니다.

➡ **[다음 슬라이드로]** 마지막 핵심 파일, **AI** 답변을 더 똑똑하게 만들어주는 **tools.py**를 보겠습니다.

슬라이드 11 · tools.py 설명

🕒 예상 시간: 1분

tools.py는 **AI**에게 자료를 손질해서 전달하는 보조 함수 모음입니다.

핵심 아이디어는 하나입니다. *****정제된 입력 → 더 정확한 답변*****입니다.

6개 함수가 각자 역할을 나눠서 담당합니다.

T01 날씨 조건 추론 — "오늘 비 오고 바람이 불어"라는 문장에서 날씨 키워드를 자동으로 감지합니다.

T02 스타일 자동 추천 — 상황 키워드로 어울리는 스타일을 판단합니다. 등교라면 캐주얼, 면접이라면 포멀처럼요.

T03 검색 키워드 생성 — 질문에서 검색에 쓸 단어를 뽑아냅니다.

T04 DuckDuckGo 검색 — 필요할 때만 실시간 검색을 실행합니다.

T05 검색 결과 요약 — 긴 검색 결과를 짧게 정리해서 **AI**에 전달합니다.

T06 입력 병합 보조 — 사용자 입력과 사이드바 설정을 합쳐서 하나로 정돈합니다.

이 흐름이 **app.py** → **tools.py** → **prompts.py** → **Ollama** 순서로 연결됩니다.

➡ **[다음 슬라이드로]** 여기서 한 가지 더, 대화를 어떻게 기억하는지 상태 관리 방식을 설명드리겠습니다.

슬라이드 12 · 통신 방식

🕒 예상 시간: 1분 30초

Python이 AI에게 요청을 보내는 방식을 설명드리겠습니다.

구조는 간단합니다. **app.py**가 클라이언트, **Ollama**가 서버입니다.

쉽게 비유하면, **app.py**가 손님이고 **Ollama**가 주방입니다. 손님이 주문서를 주방에 전달하고, 주방에서 요리해서 다시 내보내는 방식입니다.

기술적으로는 **HTTP POST** 방식으로 **JSON** 데이터를 주소 **localhost:11434**에 보냅니다. **localhost**란 내 컴퓨터 안에서 서버가 돌고 있다는 뜻입니다. 인터넷이 필요 없는 이유가 바로 이것입니다.

보내는 데이터는 4가지입니다.

- **model** — 사용할 AI 모델 이름
- **prompt** — prompts.py에서 만든 전체 지시문
- **stream** — False, 즉 답변이 완성된 뒤 한 번에 받습니다
- **options** — temperature 0.4, 창의성 수준을 조절하는 값입니다

받는 데이터는 AI가 생성한 코드 추천 본문입니다.

그리고 이 구조에서 **Streamlit**이 프론트엔드, **Ollama**가 백엔드입니다. 두 개가 HTTP로 통신하는 구조이며, **WebSocket** 같은 실시간 통신은 사용하지 않았습니다. 안정성을 우선했기 때문입니다.

➡ **[다음 슬라이드로]** 그러면 대화 기록은 어떻게 유지되는지 상태 관리 방식을 보겠습니다.

슬라이드 13 · 상태 관리 방식

🕒 예상 시간: 1분

Streamlit은 사용자가 뭔가를 입력할 때마다 페이지 전체가 새로 고쳐지는 구조입니다.

그러면 이전 대화가 다 사라져야 할 것 같은데, 실제로는 유지됩니다. 왜냐하면 **st.session_state**라는 저장소를 사용하기 때문입니다.

비유하자면, 페이지가 새로고침될 때마다 칠판이 지워지는데, `session_state`는 그 옆에 있는 메모장 같은 겁니다. 칠판이 지워져도 메모장은 그대로 남아 있습니다.

동작 방식은 세 단계입니다.

01 저장 — 사용자가 입력하면 메시지를 `session_state` 리스트에 저장합니다.

02 유지 — 페이지가 새로고침되어도 `session_state`는 초기화되지 않습니다.

03 전달 — AI를 호출할 때 최근 6개 메시지를 꺼내서 대화 맥락으로 함께 전달합니다. 덕분에 AI가 이전 대화를 기억하는 것처럼 작동합니다.

대화 초기화 버튼을 누르면 이 `session_state`를 비워서 처음 상태로 돌아갑니다.

➡ **[다음 슬라이드로]** 이제 이 프로젝트를 실제로 어떤 순서로 만들었는지 구현 과정을 말씀드리겠습니다.

슬라이드 14 · 구현 과정

🕒 예상 시간: 1분

구현은 6단계로 진행했습니다.

STEP 01 프로젝트 구조 설계 — 먼저 파일 역할을 분리했습니다. `app`, `prompts`, `tools` 세 파일로 나누는 설계부터 시작했습니다.

STEP 02 `tools.py` 구현 — 날씨 키워드 감지, 스타일 추론, 병합 함수를 작성했습니다. 재료 손질 도구를 먼저 만드는 셈입니다.

STEP 03 `prompts.py` 구현 — 8개 레이어 프롬프트를 설계하고 출력 포맷을 정의했습니다.

STEP 04 `app.py` 구현 — Streamlit UI, 사이드바, Ollama 호출, 화면 출력을 연결했습니다.

STEP 05 CSS 디자인 적용 — 블랙 × 아이보리 에디토리얼 스타일로 화면을 꾸몄습니다.

STEP 06 테스트 및 프롬프트 튜닝 — 날씨 조건별로 답변 품질을 확인하면서 레이어 규칙을 수정했습니다.

핵심은 작게 나누고, 하나씩 붙이며, 마지막에 다듬는 방식이었습니다.

➡ [다음 슬라이드로] 물론 개발 중에 문제도 있었습니다. 어떤 문제가 있었고 어떻게 해결했는지 말씀드리겠습니다.

슬라이드 15 · 문제 & 해결 과정

🕒 예상 시간: 1분 30초

개발 중에 두 가지 큰 문제가 있었습니다.

첫 번째 문제 — AI 답변에 **HTML** 코드가 섞여 출력됐습니다.

로컬 LLM이 답변을 만들 때 스타일 태그 같은 **HTML** 코드를 함께 생성하는 문제였습니다. 화면에 코디 추천 대신 `<div class="...">` 같은 코드가 그대로 보이는 상황이었습니다.

해결 — `clean_text()` 함수를 만들었습니다. 정규표현식으로 **HTML**과 **CSS** 태그를 전부 제거하고, 순수한 한국어 텍스트만 남겨서 화면에 출력하도록 했습니다.

두 번째 문제 — 사이드바 설정과 사용자 말이 충돌했습니다.

사이드바에서 "맑음"을 선택해 놓은 상태에서 채팅창에 "오늘 비 와"라고 입력하면, 어떤 날씨를 기준으로 추천해야 할지 혼동이 생겼습니다.

해결 — 사용자 입력 우선 원칙을 설계했습니다. `infer_weather_from_user_input()` 함수로 사용자 문장에서 날씨를 감지하고, `merge_user_weather_with_sidebar()` 함수로 사용자가 직접 말한 내용을 항상 우선 적용하도록 만들었습니다.

문제를 만나면 원인을 먼저 찾고, 코드로 해결한다는 원칙으로 진행했습니다.

➡ [다음 슬라이드로] 설치와 실행 방법을 간단히 안내드리겠습니다.

슬라이드 16 · 실행 방법

🕒 예상 시간: 45초

실행 방법은 8단계입니다. 터미널에서 순서대로 따라하면 됩니다.

프로젝트를 클론하고, 가상환경을 만들어 활성화합니다. 그다음 라이브러리를 설치합니다.

Ollama가 설치되어 있으면 모델을 다운받고 실행합니다. `ollama pull gemma3:4b`로 모델을 받고, `ollama run gemma3:4b`로 실행합니다.

마지막으로 `python -m streamlit run app.py`를 실행하면 브라우저에서 `localhost:8501`로 접속할 수 있습니다.

주의할 점은 Ollama가 먼저 실행 중이어야 `app.py`가 정상 작동한다는 것입니다.

➡ **[다음 슬라이드로]** 마지막으로 이 프로젝트를 통해 배운 점과 앞으로의 계획을 말씀드리겠습니다.

슬라이드 17 · 느낀 점 & 향후 개선 방향

🕒 예상 시간: 1분

이 프로젝트에서 잘 된 부분은 세 가지입니다.

로컬 LLM 기반이라 API 비용이 전혀 없고, 인터넷 없이도 동작합니다. 코드 구조를 직접 이해하면서 만들 수 있는 학습용 프로젝트이기도 했습니다.

배운 점도 세 가지입니다.

첫 번째, 로컬 LLM을 직접 연결해보니 AI가 단순히 API를 호출하는 게 아니라 그 안에 복잡한 구조가 있다는 걸 실감했습니다.

두 번째, 프롬프트 설계가 코드 작성만큼 중요하다는 걸 느꼈습니다. 규칙을 어떻게 쓰느냐에 따라 AI 답변 품질이 완전히 달라졌습니다.

세 번째, 사용자 입력을 우선하는 로직을 설계하면서 UX적인 사고방식을 배웠습니다.

앞으로의 계획은 실제 날씨 API 연동, 지역 자동 감지, 스타일 이미지 추천, 사용자 옷장 데이터 저장, 모바일 반응형 개선 순으로 진행할 예정입니다.

WEATHER FIT TALK는 단순한 챗봇이 아니라, 날씨와 상황을 이해해서 사용자에게 실용적인 스타일링을 제안하는 로컬 AI 서비스입니다.

➡ **[마지막 슬라이드로]** 이상으로 발표를 마무리하겠습니다.

슬라이드 18 · 마무리 & Q&A

🕒 예상 시간: 30초

이상으로 WEATHER FIT TALK 발표를 마치겠습니다.

오늘 발표에서 핵심 메시지 하나만 기억해 주신다면,

"로컬 **LLM**과 프롬프트 설계로, **API** 비용 없이 실용적인 **AI** 서비스를 직접 만들 수 있다"

는 것입니다.

끝까지 들어주셔서 감사합니다. 궁금한 점은 편하게 질문해 주세요.

전체 타임라인 요약

슬라이드	제목	예상 시간
01	표지	30초
02	발표 개요	30초
03	한 줄 소개	1분
04	왜 만들었나	1분
05	핵심 기능 8가지	1분
06	기술 스택	1분
07	전체 동작 흐름	1분 30초
08	파일 구조	45초
09	app.py	1분
10	prompts.py	1분 30초
11	tools.py	1분
12	통신 방식	1분 30초

슬라이드	제목	예상 시간
13	상태 관리	1분
14	구현 과정	1분
15	문제 & 해결	1분 30초
16	실행 방법	45초
17	느낀 점 & 향후 계획	1분
18	마무리 & Q&A	30초
합계		약 17분

⚠ 10~15분에 맞추려면 슬라이드 16(실행 방법)을 빠르게 넘기거나 생략하고,
슬라이드 10(prompts.py)과 15(문제 & 해결)를 각 1분으로 줄이는 것을 권장합니다.